DisasterShield AI

Master Execution Plan

Execution Handbook + Team Coordination Protocol, merged into one ordered document

TinyFish | Kritical Hacks '26 | Online round deadline: Tue 20 Oct 2026

Based on the handbook updated Fri 16 Oct 2026

Part 1. Project at a glance

This is the single plan for the four-person team. It is built directly on the official Online Round Guidelines: the 100-point judging table, the social media post tips and the TinyFish requirements. It replaces the earlier sprint plan.

1.1 What changed from the earlier plan, and why

- **Oct 20 is the deadline for the whole project, not just the PPT.** The online round needs a public GitHub repo with a README, a demo video, and a public social post link, all submitted through Unstop. The old plan had TinyFish integration in an Oct 21-26 sprint, which is after the deadline.
- **TinyFish was described wrongly.** TinyFish provides Search, Fetch, Agent (web agent) and Browser APIs for working with the live web. It is not an offline sync or edge state SDK. TinyFish is now the engine of a **Hazard Radar**, and offline behaviour comes from the PWA (service worker and IndexedDB).
- **The "Oct 31 Final MVP" is now the 24-hour finale** (31 Oct to 1 Nov, venue to be announced), only for shortlisted teams: Day 1 build and evaluation, Day 2 finals in front of the jury.
- **Revenue model needs numbers.** The jury scores "clear price economics and market research with justified prices", so the deck now has prices, market size and unit economics.

1.2 Key dates

Date	What happens
Fri 16 Oct (today)	Exams end. Kickoff, lock scope, make the repo public, get the TinyFish API key.
Sat 17 to Mon 19 Oct	Three build days (plan in Part 6).
Tue 20 Oct	Online round deadline. Submit on Unstop. The guidelines do not state a deadline time, so confirm it on the Unstop page today and submit with a buffer of several hours.
21 to 30 Oct	Screening, scoring and shortlisting by the organisers. Use this window to harden the app, deepen TinyFish usage and rehearse.
31 Oct to 1 Nov	24-hour finale at the venue (TBD). Day 1 build and evaluation rounds, Day 2 finals to an expert jury.

The core principle of the whole plan
50 of the 100 points (GitHub, TinyFish, social post) are judged on things that exist **outside the slides**. The deck alone cannot earn them. Since the deck is already done, every remaining hour goes into the repo, the TinyFish integration, the demo video and the social post.

Part 2. Scorecard: every point mapped to an owner

The jury scores out of 100. Each criterion below has a named owner and the exact evidence we will show.

Criterion	Pts	What the jury looks for	Our evidence	Owner
Working GitHub project	20	Public repo, working code, clear README, visible commits	Public repo, README that runs from a clean clone, commits from all four members across Oct 16-20	M2 (lead), all commit
TinyFish integration	20	Used meaningfully, works, relevant; explained in README and demo video	Hazard Radar worker using Search + Fetch (core) and Agent for shelters; README section with file links; 30-second demo segment	M3 (lead), M2
Social media post	10	Clear post about the project, public, GitHub link, demo video/GIF	LinkedIn post with GIF and repo link (draft in Part 12)	M4
Technique	10	Quality of technical approach, architecture, execution	Architecture slide, clean contracts, working offline queue and sync	M2, M1
Innovation and idea	20	How original the problem and solution are	Live-web radar + offline-first AI photo verification in one flow	M4, M3
Impact and usefulness	10	Real problem, real people, realistic	Scenario slide, measured pilot metrics from our own demo run	M4
Revenue model	10	Clear price economics and market research with justified prices	Revenue slide: prices, TAM/SAM/SOM, unit economics, sources (slide already complete)	M4

Note
50 of the 100 points (GitHub, TinyFish, social post) are judged on things that exist outside the slides. The deck alone cannot earn them.

Part 3. Scope: what we build in four days

Feature	Priority	Reason
TinyFish Search + Fetch powering the Hazard Radar and alerts feed	**MUST**	Worth 20 points and the mandatory rule of the hackathon
Citizen PWA: alerts, map, shelters, cached for offline use	**MUST**	Core of the offline-first claim
SOS with offline queue, sync on reconnect, mocked SMS fallback	**MUST**	The most memorable demo moment
Image verification endpoint with a pretrained model and confidence score	**MUST**	Differentiator for innovation and technique
Authority dashboard map with incidents and a simple grid heatmap	**MUST**	Shows the second user group
TinyFish Agent collecting shelter data for one district	STRETCH	Deepens the TinyFish story; costs credits, so run once and cache
TinyFish Browser API for JavaScript-heavy dashboards	STRETCH	Only if Fetch cannot render a needed page
Training your own YOLOv8 model, on-device edge AI, drones, IoT, digital twin	CUT	Future scope slide only. Use a pretrained model and say so honestly in the README.
Real Twilio SMS	CUT	Mock the SMS gateway and say so in the README and video

Rule for the whole team
If something is mocked, **label it as mocked** in the README and in the video. The jury scores "works" and "realistic"; honest labels protect both.

If time runs short, cut STRETCH items first. The MUST list above is the minimum submission.

Part 4. Team roles and boundaries

Four members, clear lanes. M4 is now a hybrid: product and delivery owner **plus** a technical helper to M1, M2 and M3 because the deck is finished.

4.1 Member 1 (M1): Frontend and UI lead

React.js, Tailwind, Leaflet, Service Worker

- Citizen app (mobile-first) with three taps: Alerts, Map, SOS. Authority dashboard (desktop web).
- Leaflet map with shelters, danger zones and the heatmap layer from M2's grid endpoint.
- Service Worker caching for the app shell, API responses and map tiles. IndexedDB queue for SOS and reports, with a sync call on reconnect.
- Show "last updated" time and source link on every alert.
- **Boundary:** no backend routes or SQL. Use mock JSON until M2's endpoints are live. Does not change the database schema.
- Co-director duty with M4 (see 4.5).

4.2 Member 2 (M2): Backend and database architect

Node.js, Express, PostgreSQL, PostGIS

- Schema, migrations and seed data. All endpoints in the contract table (Part 5). JWT and role-based access.
- SOS endpoint, offline sync endpoint (idempotent using a client-generated id) and mock SMS fallback.
- Owns the repo, branch rules and README structure. Reviews pull requests.
- Stretch: TinyFish Agent script to collect shelter data for one district into the shelters table.
- **Boundary:** no CSS or React. Does not own the radar worker or model code; consumes M3's output through the alerts table and the verify endpoint.

4.3 Member 3 (M3): AI and GIS specialist

Python, TinyFish SDK, pretrained vision model

- Hazard Radar worker (the main TinyFish integration): Search for fresh alerts, Fetch official pages, extract structured alerts, write them to the database, and save raw responses as snapshots.
- Image verification service: pretrained model, returns label and confidence; documents its limits honestly.
- Simple severity logic and district matching so alerts reach the right users.
- **Boundary:** no main web server. Exposes capabilities as a worker plus one HTTP endpoint that M2 calls. Does not edit the frontend.

4.4 Member 4 (M4): Product, delivery and technical support [REVISED]

Figma, README, video, QA, plus React, API testing and data support

> **Why M4's role changed**
> The PPT/pitch deck is complete. The original handbook gave M4 deck review, deck fact-checking and slide work. That time is now reassigned to technical work so M4 directly helps the three developers finish the MUST list. The Coordination Guide already required M4 to "take on technical work" and help M1; this plan extends that to M2 and M3 as well.

A. Kept from the original role (product and delivery)

- Owns the narrative against the scorecard: README top section, demo video script and edit, social post.
- QA: runs the full offline-to-online scenario each evening and logs bugs.
- Opens the Unstop page early, notes the exact deadline time and submission fields, keeps the shared folder for screenshots and clips.
- Collects 10 flood/damage test images with usable licences, captures screenshots and the offline SOS GIF.
- Revenue model facts: the deck is done; M4 only updates the last slide with the three real links before submission.
- Co-director duty with M1 on the 9 PM feature freeze (4.5).

B. NEW technical support duties

[NEW] Support for M1 (Frontend)

- Build React UI components from M1's mockData.json: dashboard layout, SOS button styling, alert cards, shelter list, cached/offline status banner.
- Build the authority dashboard screen directly in code (not only in Figma), including the photo upload box and the AI confidence badge.
- Agree a folder (for example web/src/components/) with M1 so the two do not edit the same files. Work on short feature branches.

[NEW] Support for M2 (Backend)

- Prepare seed data files (alerts, shelters, sample incidents) as JSON/CSV for M2 to load, and the .env.example and mock-mode setup docs.
- Write a Postman or Thunder Client collection plus a small smoke-test script for every endpoint in Part 5, especially the idempotency check on POST /api/sos/trigger and POST /api/sync (send the same clientId twice, expect no duplicate).
- Help build the mock SMS fallback log view so the demo can show "SMS sent (mocked)" entries.
- Open pull requests only; M2 reviews and merges.

[NEW] Support for M3 (AI and GIS)

- Research and list three official sources per hazard (cyclone, flood, rainfall, earthquake) for each monitored district, to feed the radar and the "government site changes layout" fallback.
- Write test cases for extract.py: sample bulletin texts and the expected {hazard, severity, district, validUntil, sourceUrl} output.
- Run the collected test images through M3's verify endpoint, log label and confidence in a table, and flag poor results so the README can state the model's limits honestly.
- Help organise the snapshots/ folder and the sample TinyFish response screenshot for the README.

C. M4 boundary (unchanged)

M4 does not push to the main branch without developer review. All of M4's code goes through feature branches and pull requests reviewed by M2.

4.5 M1 and M4 Co-Director Duty

M1 and M4 share the responsibility of steering the final project. On **Oct 19**, they must jointly enforce the **9 PM Feature Freeze**. Together, M1 and M4 decide if an incomplete feature gets deleted or replaced with a mock so the video recording can begin immediately.

4.6 Responsibilities at a glance

Member	Primary lane	Helps with
M1	React PWA, Leaflet, service worker, IndexedDB queue, dashboard UI	Receives UI help from M4
M2	Node, Express, PostGIS, JWT, sync, repo owner, PR reviewer	Receives seed data, tests and SMS log view from M4
M3	TinyFish radar, extractor, image verification	Receives source lists, extractor tests and image evaluation from M4
M4	README, video, social post, QA, Unstop submission, Oct 20 link updates	Gives technical help to M1, M2 and M3

Part 5. API contracts

Agree these on the kickoff call. **Do not change them without telling the team.** They are what lets everyone work in parallel (see Part 7).

Endpoint	Owner	Purpose and shape
GET /api/alerts?lat&lng&radius	M2	Returns alerts near a point: hazard, severity, district, validUntil, sourceUrl, fetchedAt
GET /api/shelters?lat&lng	M2	Nearest shelters with name, location, capacity
POST /api/sos/trigger	M2	SOS with clientId, lat, lng, timestamp. Idempotent on clientId
POST /api/sync	M2	Flush the offline queue (SOS and reports) in one call
POST /api/reports	M2	Photo report with location. Calls the verify endpoint and stores the score
POST /api/ai/verify-image	M3	Returns label and confidence for one image
GET /api/heatmap	M2	Grid cells with incident counts for the dashboard

Mock reply for testing (from the Coordination Guide): until M3's model is ready, M2 hardcodes the verify endpoint to immediately return:

```
{"confidence": 0.85, "label": "flood"}
```

Part 6. Day-by-day build plan (Oct 16 to Oct 20)

Tags: **[NEW]** means added for M4's technical role; **[CHANGED]** means the original task changed because the PPT is done. Follow the Daily Rhythm in Part 8 every day.

Fri 16 Oct (tonight, 1 to 2 hours)

Who	Tasks
All	30-minute kickoff. Agree the scope (Part 3) and the API contracts (Part 5).
M2	Create the GitHub repo, set it to public, add a README skeleton, .gitignore and .env.example. Invite the other three. **Never commit real keys.**
M3	Create a TinyFish account at agent.tinyfish.ai (use the link in the guidelines for the free credits) and an API key. Run the first Search and Fetch call from the quick-start.
M4	Open the Unstop page, note the exact deadline time and the submission fields, and create a shared folder for screenshots and video clips. **[NEW]** Clone the repo, set up the local dev environment, and agree the component folder split with M1.
M1	Initialise the React + Tailwind PWA with routing and push it.

Sat 17 Oct: Build day 1 (foundations that run)

Mock-first rule is active from today: nobody waits for anyone (Part 7).

Who	Deliverables by end of day
M1	Citizen app shell with three screens (Alerts, Map, SOS). Leaflet map rendering mock shelters. Service worker registered and caching the app shell. Built against a local mockData.json (3 alerts, 4 shelters, map points).
M2	PostGIS schema deployed (users, alerts, shelters, incidents, sos_events). GET /api/alerts and POST /api/sos/trigger returning real DB rows. JWT with two roles. Hardcoded fake verify response so frontend report testing is not blocked.
M3	Radar worker v0: Search for latest alerts on a short list of districts, Fetch the top official pages, save raw JSON snapshots with timestamps. Pretrained image model running locally on three test photos. Output written to local .json/.csv files first; M2 consumes them when ready.
M4	**[CHANGED]** Final deck review not needed, the PPT is complete (do a 10-minute check of the deck against the scorecard only). README outline. Demo storyboard (Part 11). Collect 10 flood/damage test images with usable licences. **[NEW]** With M1: build the SOS button, alert card and shelter list components from mockData.json. **[NEW]** With M2: prepare seed JSON/CSV and the first Postman/Thunder collection for the contracts. **[NEW]** With M3: start the list of three official sources per hazard per district.

Sun 18 Oct: Build day 2 (connect the pieces)

Who	Deliverables by end of day
M1	App reads /api/alerts and /api/shelters. **Offline mode works:** turn off the network and alerts, shelters and map tiles still load. SOS button writes to an IndexedDB queue.
M2	POST /api/sync (idempotent flush of the offline queue), mock SMS fallback logging to DB, POST /api/reports calling M3's verify endpoint. Heatmap query (grid counts of incidents).
M3	Radar worker v1: extraction turns fetched text into structured alerts (hazard, severity, district, valid-until, source URL) and inserts them through M2's table. POST /api/ai/verify-image returns label and confidence.
M4	Authority dashboard layout in Figma or directly with M1. Start the QA checklist. **[CHANGED]** Pricing and market slides fact-checked against sources not needed, the deck is complete (only keep a sources note for the README if useful). **[NEW]** With M1: build the authority dashboard screen in React (map area, incident list, photo upload box, confidence badge). **[NEW]** With M2: smoke tests for /api/sos/trigger and /api/sync idempotency; mock SMS log view. **[NEW]** With M3: extract.py test cases and a table of image results (label, confidence) for the README limits section.

Mon 19 Oct: Build day 3 (end-to-end and freeze)

Morning, M1 + M2 + M3: run the full scenario end to end: radar creates alert, phone shows it, go offline, SOS queued, reconnect, sync, dashboard shows cluster and verified photo.

Who	Tasks
M3 (afternoon, stretch)	TinyFish Agent run to collect shelter data for one district into the shelters table. **Cache the result; do not run it live during the demo.**
M4 (afternoon)	README complete (Part 10), record rough demo takes, capture screenshots and a short GIF of the offline SOS flow. **[NEW]** Run the full QA scenario and log bugs for the right owner (M1, M2 or M3). Help M1 with the heatmap layer styling and the dashboard polish. Help M3 re-check image results after any model changes.
M1 + M4	**9 PM feature freeze.** After this, only bug fixes and polish. Co-directors decide whether an unfinished feature is deleted or replaced with a mock.

Tue 20 Oct: Submission day

- **Morning:** fix bugs, fresh clone test (Part 10), final screenshots.
- Record and upload the final demo video (Part 11). Publish the social post with the video or GIF and the repo link (Part 12).
- Open the repo and the post in a **private browser window** to confirm both are public.
- **Submit on Unstop several hours before the deadline.** Screenshot the confirmation.
- **[NEW]** M4: update the last slide of the (already finished) deck with the real repo, video and post links before submitting.

Part 7. Anti-blocking strategy (Mock First)

The core problem addressed
Team members are often blocked because one person's incomplete work ruins the next person's task. In a 4-day sprint, sequential waiting will cause project failure. The solution is **decoupling via strict API contracts** (Part 5). To execute the timeline by Oct 20, the team must shift from "waiting for real integration" to "developing in parallel using mocked data."

No member is allowed to use the excuse "I am waiting for [Member] to finish." Every dependency mapped in the plan must be bypassed using fake data on Day 1 (Oct 17). Integration only happens when both sides are 100% ready.

Role	What usually blocks them	The mandatory "Unblock" action
M1 (Frontend)	Waiting for M2 to finish building the PostgreSQL database and Express routes to display alerts or map data.	Create a mockData.json file locally. Hardcode 3 alerts, 4 shelters, and map points. Build the entire UI using this JSON file.
M2 (Backend)	Waiting for M3 to finish the Python Vision model or the TinyFish SDK integration to test data flows.	Do not wait for Python. Hardcode an endpoint that immediately returns a fake {"confidence": 0.85, "label": "flood"} to test frontend reports.
M3 (AI/GIS)	Waiting for M2 to finalize database tables before saving TinyFish JSON outputs.	Write output strictly to local .json or .csv files first. M2 will consume these when ready. Focus entirely on TinyFish logic.
M4 (PM/QA/UI)	Waiting for the app to be fully integrated to take screenshots or write the README.	Since the PPT is complete, M4 must take on technical work. Assist M1 by building React UI components (like the dashboard layout or SOS button styling) using M1's mocked data. **[NEW]** Also assist M2 (seed data, endpoint tests) and M3 (source list, extractor tests, image evaluation), as in Part 4.4.

By enforcing this rule, if M2 falls behind on Sunday, M1 still finishes the entire frontend, and M4 can still record a mostly-working demo video.

Part 8. The daily hackathon rhythm

To prevent silent failures where someone does not do their work without the team knowing, enforce these touchpoints every day.

Time	Meeting / Action	Mandatory format
10:00 AM	15-Minute Standup	Each member answers: 1) What I finished yesterday. 2) What I am doing today. 3) What is blocking me right now? (Be honest if struggling).
4:00 PM	Mid-Day Checkin	A quick text/Slack update from everyone. Example: "M1: UI done. M2: Database seeded." Catch failures early before the night block.
9:00 PM	The Integration Handshake	If M1 and M2 are ready, replace the mock data with the real API endpoint. If one is not ready, push the handshake to tomorrow morning. **Do not stay up all night breaking working mocks.**
11:00 PM	Daily Code Push	All working code, even if it just runs mock data, **MUST** be pushed to GitHub. M2 reviews pull requests. Never leave code isolated on one laptop.

How this connects to the rest of the plan

The 9 PM handshake runs every evening; on Mon 19 Oct the 9 PM **feature freeze** also applies (Part 6). The 11 PM push also builds the visible commit history from all four members that the 20-point GitHub criterion looks for. **[NEW]** M4's QA run each evening should happen before the 9 PM handshake so bugs reach owners in time.

Part 9. TinyFish integration spec (20 points)

The jury wants TinyFish "used meaningfully, works, and relevant to your use case", explained in the README and the demo video. A token or decorative use scores poorly, per the guidelines FAQ.

API	Endpoint (from the TinyFish docs)	How we use it	Status
Search	GET https://api.search.tinyfish.ai SDK: client.search.query(query=...)	Find the latest cyclone, flood, rainfall and earthquake alerts for each monitored district. Supports a news-only mode.	MUST
Fetch	POST https://api.fetch.tinyfish.ai SDK: client.fetch.get_contents(urls=[...])	Render the official bulletin pages found by Search and return clean text for the extractor.	MUST
Agent	POST https://agent.tinyfish.ai/v1/automation /run-sse (also /run, /run-async)	Goal-based task on a portal with no API: collect shelter names, addresses and capacity for one district.	STRETCH
Browser	POST https://api.browser.tinyfish.ai	Playwright/CDP session for a JavaScript-heavy page that Fetch cannot render.	STRETCH

- **Auth:** header X-API-Key with the key from agent.tinyfish.ai/api-keys. Keep it in TINYFISH_API_KEY in a local .env. **Never commit it.**
- **Cost:** per the docs, Search and Fetch do not use credits; Agent and Browser do. Build the radar on Search and Fetch so we do not run out of free credits, and run Agent once, then cache.
- **Docs:** docs.tinyfish.ai (the page docs.tinyfish.ai/llms.txt lists every page).

Radar worker folder structure

```
radar/
  run.py                # loop: search > fetch > extract > insert alerts
  tinyfish_client.py    # thin wrapper around Search and Fetch (timeouts, retries)
  extract.py            # text > {hazard, severity, district, validUntil, sourceUrl}
  snapshots/            # raw TinyFish responses with timestamps (proof and demo fallback)
```

Make TinyFish visible to the jury

- README section "How TinyFish is used" with links to the exact files and a screenshot of a real response.
- Every alert in the app shows its source link and fetched time, so TinyFish's output is visible to the user.
- In the video, show the radar log fetching a live page, then the same alert appearing on the phone (Part 11).
- Keep snapshots so the demo still works if the network or a government site is slow on the day.
- **[NEW]** M4 supports M3 with the official-source list and extract.py test cases (Part 4.4).

Part 14. Risks, fallbacks and submission-day failsafes

14.1 Risk table

Risk	Fallback	Owner
A government site changes layout or blocks a request	Use saved snapshots; keep a list of three sources per hazard	M3 (M4 helps with the source list)
Free TinyFish credits run low	Radar runs on Search and Fetch only; run Agent once and cache	M3
Vision model gives poor results	Show confidence honestly; present as assistive, with a human in the loop	M3
Offline sync bug during recording	Test nightly; keep a recorded clean take	M1, M4
Time runs short	Cut STRETCH items first. The MUST list in Part 3 is the minimum submission.	M2
Links not public at judging time	Private-window check before submit	M4
A teammate silently stalls	10 AM standup and 4 PM checkin make blockers visible early (Part 8)	All

14.2 Failsafes for submission day (Oct 20)

The scoring is heavily weighted toward TinyFish integration and a working GitHub repo. If integration completely fails between members, invoke these failsafes:

Failsafe A: The M3/M1 bypass	Failsafe B: Honest mocks
If M2's backend fails entirely, M1 (Frontend) should modify the React app to directly read the static JSON files that M3 (Python/TinyFish) generates. This skips the database but keeps the TinyFish points intact for the demo.	If the AI Vision model (M3) breaks, keep M2's fake response code. M4 must state in the demo video: "Due to time limits, the AI classification endpoint is returning mock scores, but the end-to-end data flow is active." The jury respects transparency over broken code.

M1 and M4 Co-Director Duty
On Oct 19, M1 and M4 jointly enforce the 9 PM Feature Freeze. Together they decide if an incomplete feature gets deleted or replaced with a mock so the video recording can begin immediately.

Part 15. Final submission checklist (Tue 20 Oct)

- [ ] Repo public, README complete, fresh clone runs with the documented commands
- [ ] TinyFish section in README with file links and a sample response
- [ ] No API keys in the repo or in its commit history
- [ ] Demo video uploaded and playable; TinyFish clearly shown
- [ ] Social post public with GIF or video and the GitHub link
- [ ] Deck updated with the three real links on the last slide (deck itself already complete)
- [ ] All members listed on Unstop; submission confirmed and screenshotted
- [ ] Anything mocked is labelled in the README and the video
- [ ] Submitted several hours before the deadline time shown on Unstop

Sources

Kritical Hacks '26 Online Round Guidelines and FAQ (organisers); TinyFish developer documentation at docs.tinyfish.ai (endpoints and credit usage as published there; confirm details on the live docs when integrating). Merged from: DisasterShield AI Execution Handbook (updated Fri 16 Oct 2026) and DisasterShield AI Team Coordination Guide.